National University of Computer & Emerging Sciences

Karachi Campus

CS4051 - INFORMATION RETRIEVAL

ASSIGNMENT 1 - BOOLEAN RETRIEVAL SYSTEM

MUHAMMAD HASAN SAMI

23i-0710

BCS-6A

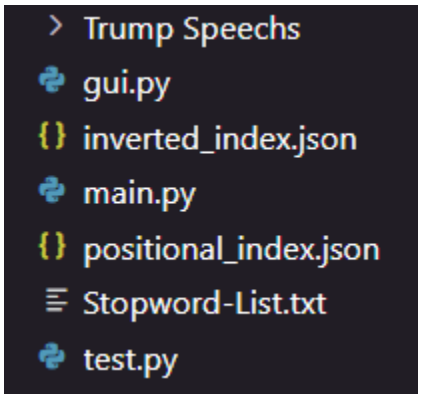
Introduction

This project is a Boolean Retrieval Model built to search through speeches of Donald Trump. The goal of the system is to help users quickly find relevant speeches or documents based on specific keywords or combinations of words.

Instead of scanning entire speeches manually, the system processes the text, removes unnecessary words (using a list of stopwords in *stopwords.txt),* stems the remaining valid tokens using the **Porter Stemmer,** and builds an inverted index of the stemmed terms. It also builds a positional index to support proximity queries. Users can perform searches using Boolean queries such as AND, OR, and NOT to retrieve precise results.

For example, a user can search for terms like “economy AND jobs” to find speeches where both topics are discussed, or “immigration NOT border” to exclude certain contexts. The system also supports proximity search, so the user can query “United America /2”, which will retrieve all documents where the two words are found within 2 positions of each other.

File Structure



- **Trump Speechs:** The folder containing all 57 documents to be indexed.
- **gui.py:** Implements the Graphical User Interface built on Qt.
- **Inverted_index.json:** The inverted index mapping all the stemmed terms to the documents they appear in.
- **main.py:** Contains the core logic and the CLI.
- **Positional_index.json:** The positional index storing the raw positions of all terms.
- **Stopword-List.txt:** A file containing all stopwords.
- **test.py:** A dummy file to compare actual and expected results

How To Run

Simply run the following command to install PyQt5 and NLTK libraries:

```
py -m pip install -r requirements.txt
```

Then run `py main.py` to run on the CLI or `py gui.py` to run the GUI. The CLI is as follows:

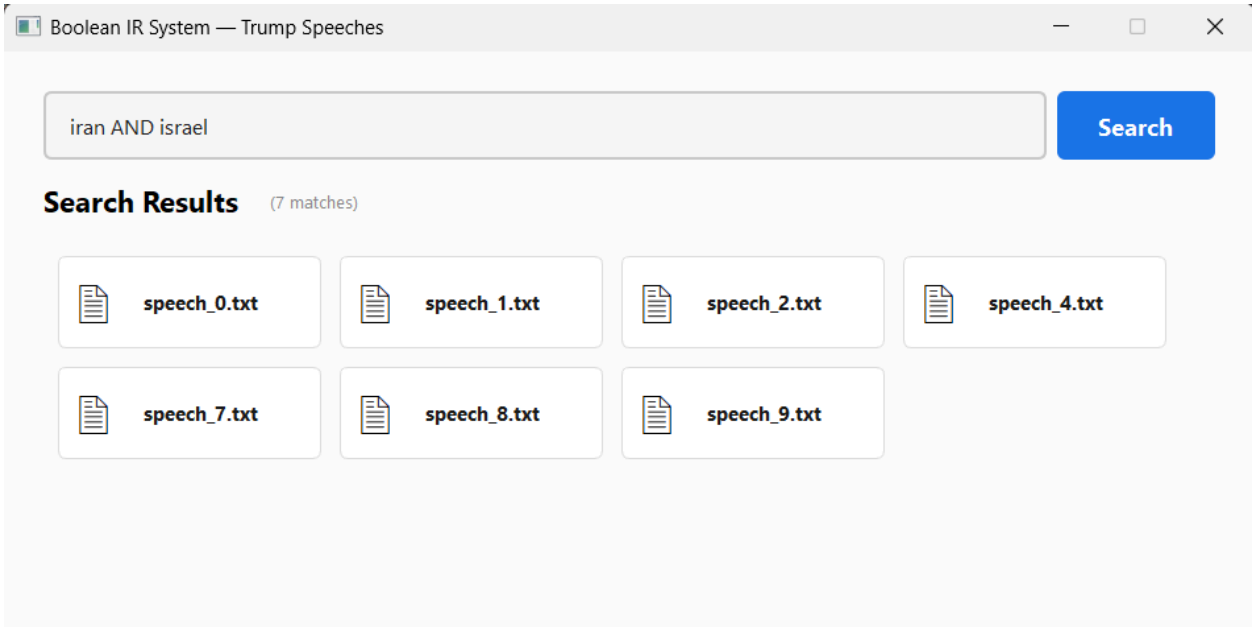
```
[INFO] Loaded indexes from disk.
==============================================================
                Boolean IR System — Trump Speeches
==============================================================
Query formats:
  t1 AND t2 AND t3
  t1 OR  t2
  NOT t1
  t1 AND NOT t2
  t1 t2 / k (proximity: t1 and t2 within k words)
  Type 'exit' to quit.

Enter query: running

Query: running
{'52', '10', '24', '5', '6', '40', '51', '9', '1', '17', '30', '45', '34', '33', '18', '37', '39', '47', '50', '12', '20', '
16', '28', '21', '22', '27', '2', '0', '32', '41', '35', '8', '36', '26', '53', '3', '19', '44', '46', '11', '25', '4'}
(42 document(s) matched)
Enter query:
```

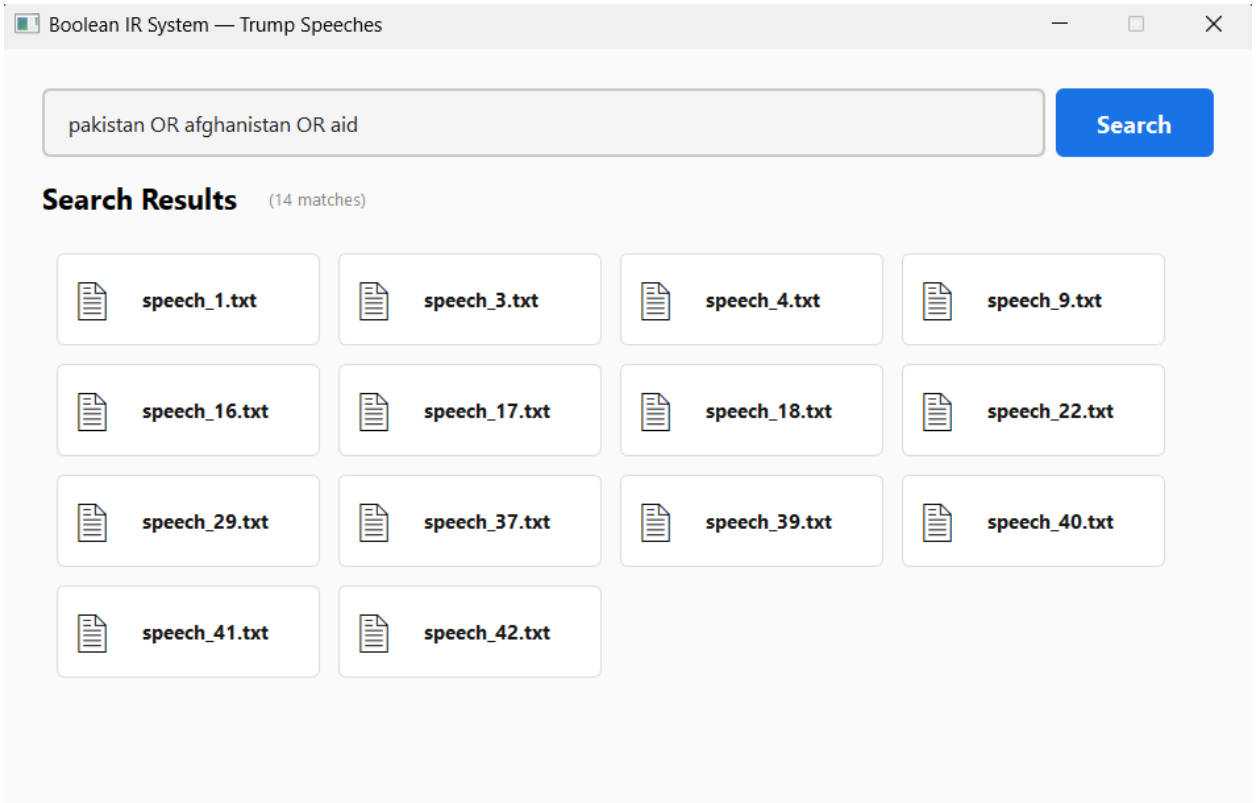
The program checks to see if indexes are present on disk. If not it will first build the indexes.

Boolean Query

This was relatively straightforward. An AND / OR query meant using Python's built-in set operators &/| to construct a resultant set. A NOT query is executed by doing ALL - s (subtract the indexed term set from the entire set).

The complicated part was handling the parentheses and subqueries. The program therefore checks for parentheses and in such a case it first evaluates the subquery and then merges (or injects) it to the main query. A limitation of the program is that it only handles one level of parentheses, so any further nesting would provide inaccurate results.


Boolean IR System — Trump Speeches
iran AND israel
Search
Search Results
(7 matches)
speech_0.txt
speech_1.txt
speech_2.txt
speech_4.txt
speech_7.txt
speech_8.txt
speech_9.txt


Boolean IR System — Trump Speeches
pakistan OR afghanistan OR aid
Search
Search Results
(14 matches)
speech_1.txt
speech_3.txt
speech_4.txt
speech_9.txt
speech_16.txt
speech_17.txt
speech_18.txt
speech_22.txt
speech_29.txt
speech_37.txt
speech_39.txt
speech_40.txt
speech_41.txt
speech_42.txt

Proximity Query

The query format:

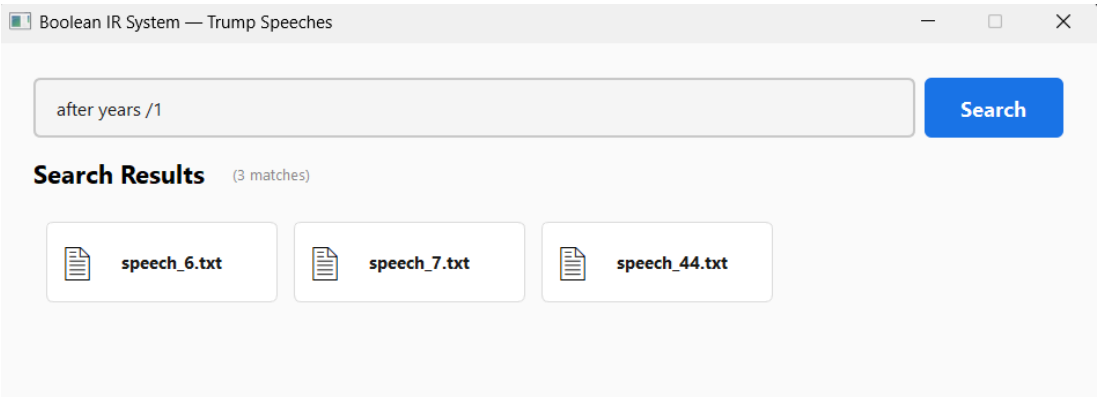
t1 t2 /k, where k = number of words between t1 and t2.

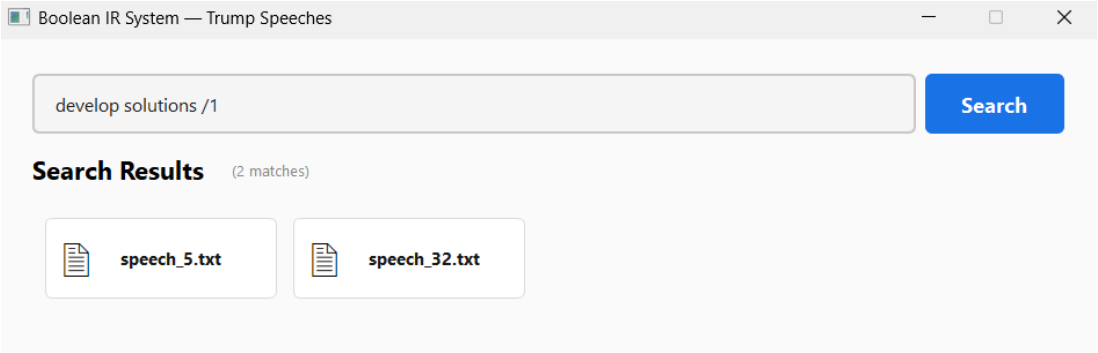
[NOTE] Although a proximity query generally returns documents where t1 and t2 have a distance that is **LESS THAN OR EQUAL TO** k, the requirement stated that our query should only return documents where t1 and t2 have a distance that is **EXACTLY EQUAL TO** k, meaning there should be exactly k words between the two terms. The test cases provided also confirm that.

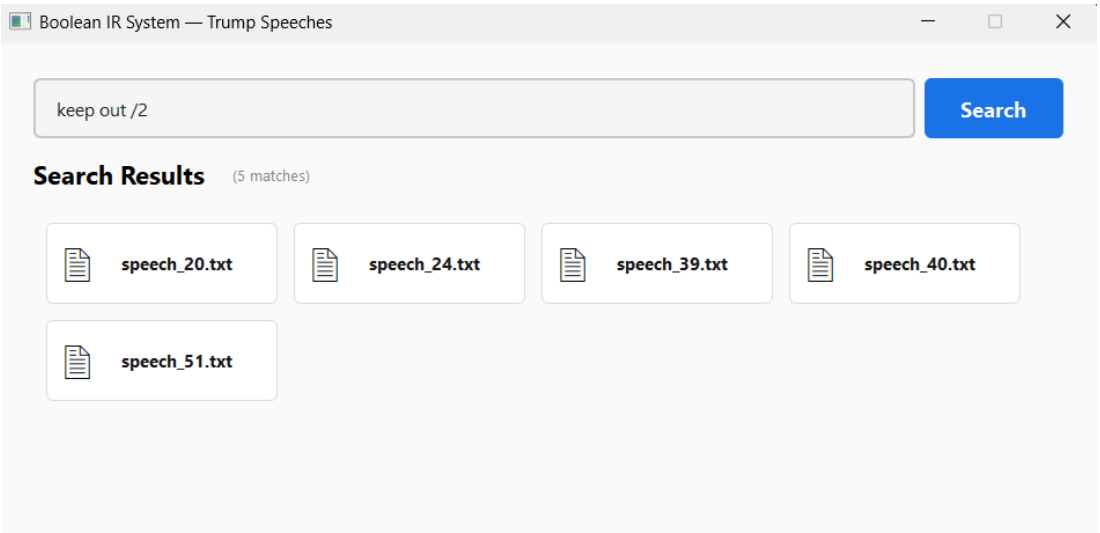
Another issue was that since preprocessing removes stopwords, they do not end up in the positional index and are therefore ignored if they appear between t1 and t2. This messes up the distance calculation. Therefore, even though I did not include the stopwords in my inverted index, I still had to store their positions in the positional index.

The same issue occurred with processing numbers. For example, the query “after years /1” ignored the result “after 12 years” because the number 12 was removed during preprocessing. Hence I had to also provide a position for numbers.

The outputs of all the proximity test cases:







Error Handling

The GUI gracefully handles errors by using try/except blocks and outputting any raised exceptions or errors from main:

AI Assistance

AI was used to learn about JSON-based indexing, and how to build and load indexes in Python using json.dump(). It was also used to debug the proximity query function, where my implementation had failed due to an incorrectly incrementing j counter. It was also used to add parentheses logic in the boolean query, and writing the skeletal code for Qt components - QtWindow and the DocCard.

Conclusion

In conclusion, this project provided a strong understanding of how information retrieval systems work, particularly the role of indexing in efficient text search. I learned how raw text is transformed into a structured format that enables fast and accurate querying.

One of the key learnings was the difference between a basic inverted index and a positional index. While an inverted index simply maps terms to the documents they appear in, it is limited when it comes to handling more advanced queries such as proximity searches. In contrast, a positional index stores the exact positions of terms within documents, making it possible to accurately determine how close two terms appear to each other.

During implementation, I also encountered practical challenges, particularly with preprocessing steps like stopword removal. This highlighted an important concept: while removing stopwords improves efficiency, it can negatively affect positional accuracy if not handled carefully. By preserving original positions even after filtering terms, I was able to resolve proximity-related issues and better understand the trade-offs involved in designing retrieval systems.